K.Shamela

Mini-Project : 5

Project Title : Comment Toxicity Detection

Course :Data Science

Batch : DS-C-WD-E-B144

Comment Toxicity Detection

Problem Statement:

Online communities and social media platforms have become integral parts of modern communication, facilitating interactions and discussions on various topics. However, the prevalence of toxic comments, which include harassment, hate speech, and offensive language, poses significant challenges to maintaining healthy and constructive online discourse. To address this issue, there is a pressing need for automated systems capable of detecting and flagging toxic comments in real-time.

The objective of this project is to develop a deep learning-based comment toxicity model using Python. This model will analyze text input from online comments and predict the likelihood of each comment being toxic. By accurately identifying toxic comments, the model will assist platform moderators and administrators in taking appropriate actions to mitigate the negative impact of toxic behaviour, such as filtering, warning users, or initiating further review processes.

Introduction:

With the rapid growth of social media platforms, online forums, and digital communication, large amounts of user-generated content are created every day. While these platforms encourage communication and information sharing, they also contain harmful and abusive comments that can negatively impact users and online communities.

Manual moderation of toxic comments becomes difficult due to the huge volume of online data. Therefore, automated toxic comment detection systems are important

for identifying harmful content efficiently and maintaining healthy online environments.

This project focuses on detecting toxic comments using Natural Language Processing (NLP) and Deep Learning techniques. The system classifies comments into multiple toxicity categories such as toxic, severe toxic, obscene, threat, insult, and identity hate.

The project uses text preprocessing techniques, Exploratory Data Analysis (EDA), and a Bidirectional LSTM deep learning model to learn meaningful text patterns from comments. An interactive Streamlit application was also developed to allow users to perform real-time toxicity prediction and bulk CSV predictions.

The main objective of this project is to build an intelligent system that can automatically identify toxic comments and support safer online communication platforms.

Objective

The main objective of this project is to develop an automated toxic comment detection system using Deep Learning and NLP techniques.

The system should accurately classify toxic comments and help improve online communication safety by identifying harmful content in real-time.

Dataset:

Toxic Comment Classification Dataset

Dataset Details:

- Total comments: 159,571
- Multi-label classification dataset
- Categories:

    - Toxic
    - Severe Toxic
    - Obscene
    - Threat
    - Insult
    - Identity Hate

Data Preprocessing

The following preprocessing steps were performed:

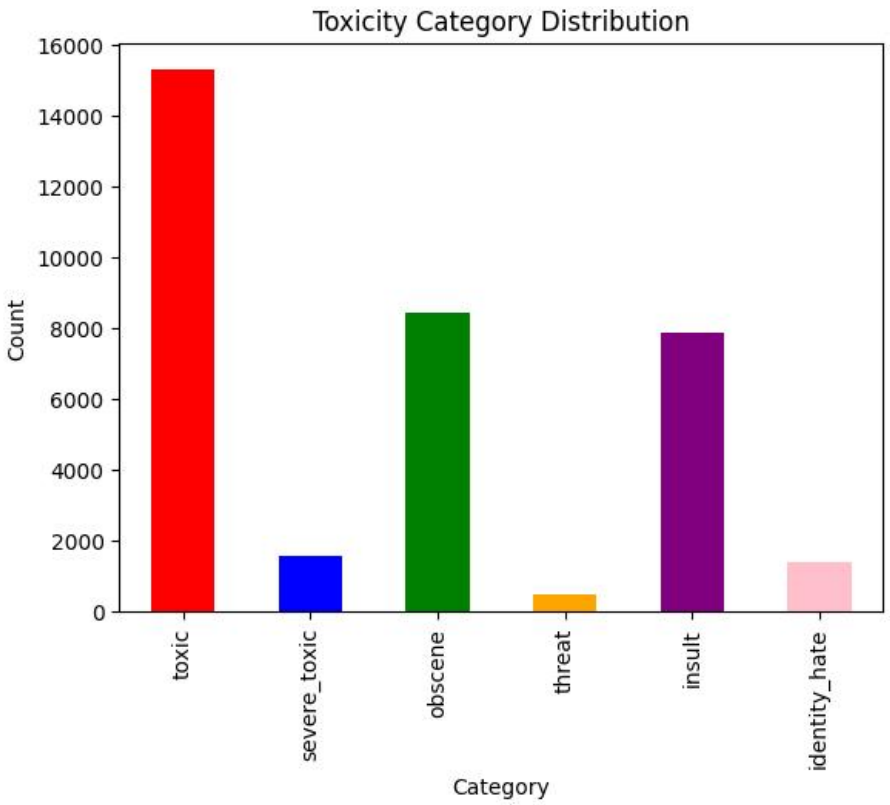
- Lowercasing text
- Removing URLs
- Removing punctuation
- Removing stopwords
- Removing special characters
- Tokenization
- Text cleaning using Regular Expressions
- Text vectorization

The cleaned text improved data quality and helped the model learn meaningful patterns more effectively.

Exploratory Data Analysis (EDA)

EDA techniques used in the project include:

- Toxicity category distribution analysis

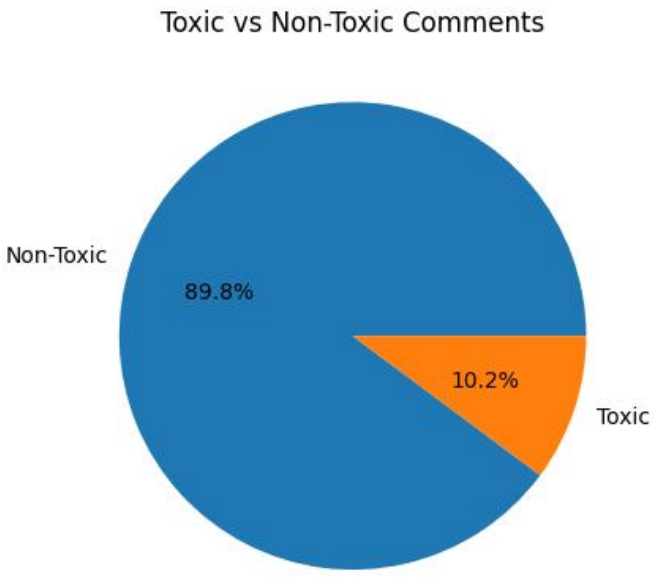


This bar chart shows the distribution of toxicity categories in the dataset.

We can observe that the 'toxic' category has the highest number of comments, while categories like 'obscene' and 'insult' have comparatively fewer samples.

This indicates that the dataset is imbalanced, which is common in real-world classification problems.

- Pie chart visualization

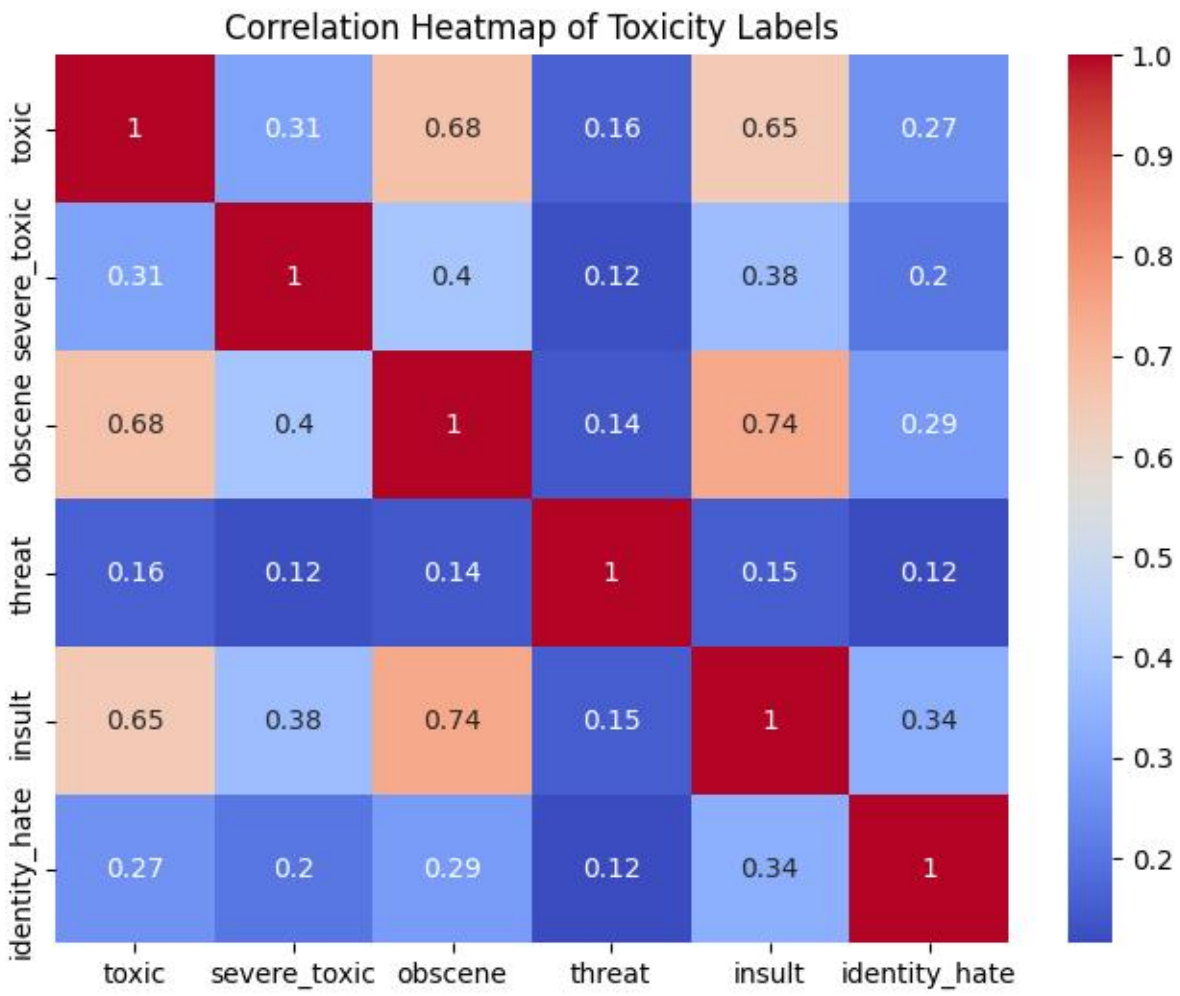


The pie chart shows the distribution of toxic and non-toxic comments in the dataset.

Most comments belong to the non-toxic category, while toxic comments form a smaller portion of the dataset.

This imbalance is common in real-world text classification problems.

- Heatmap correlation analysis

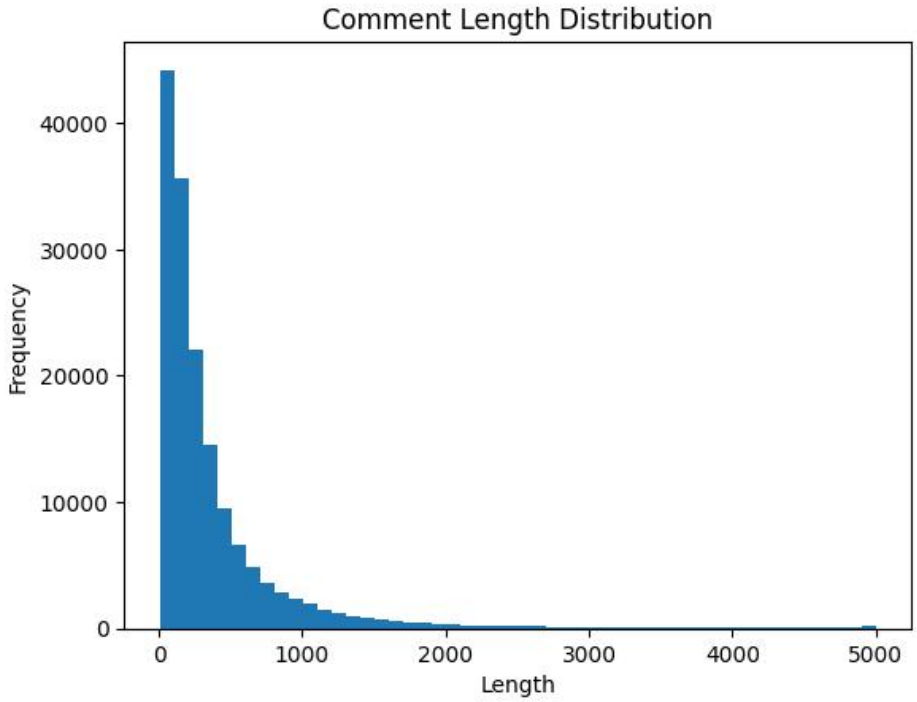


The heatmap shows the correlation between different toxicity labels.

We observe that some categories such as 'toxic', 'obscene', and 'insult' are highly correlated, meaning they often occur together in comments.

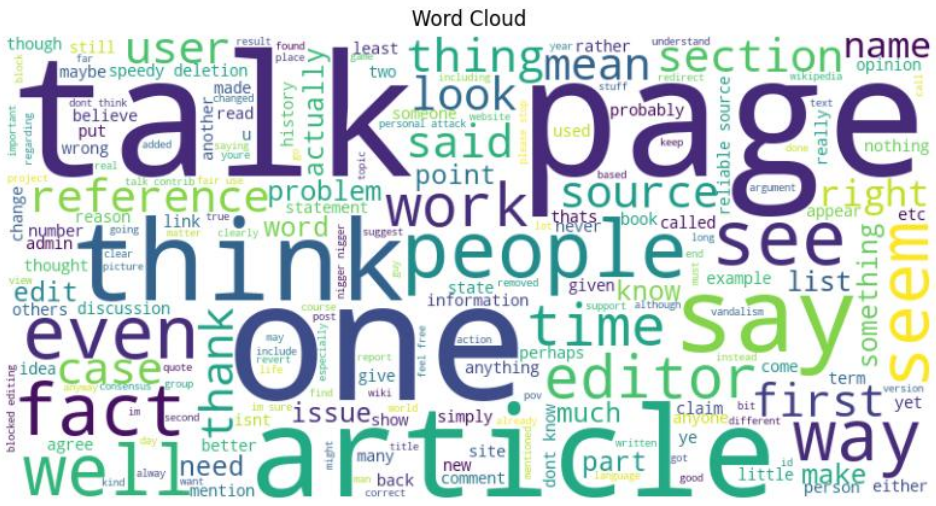
Lower correlation values indicate weaker relationships between certain toxicity classes.

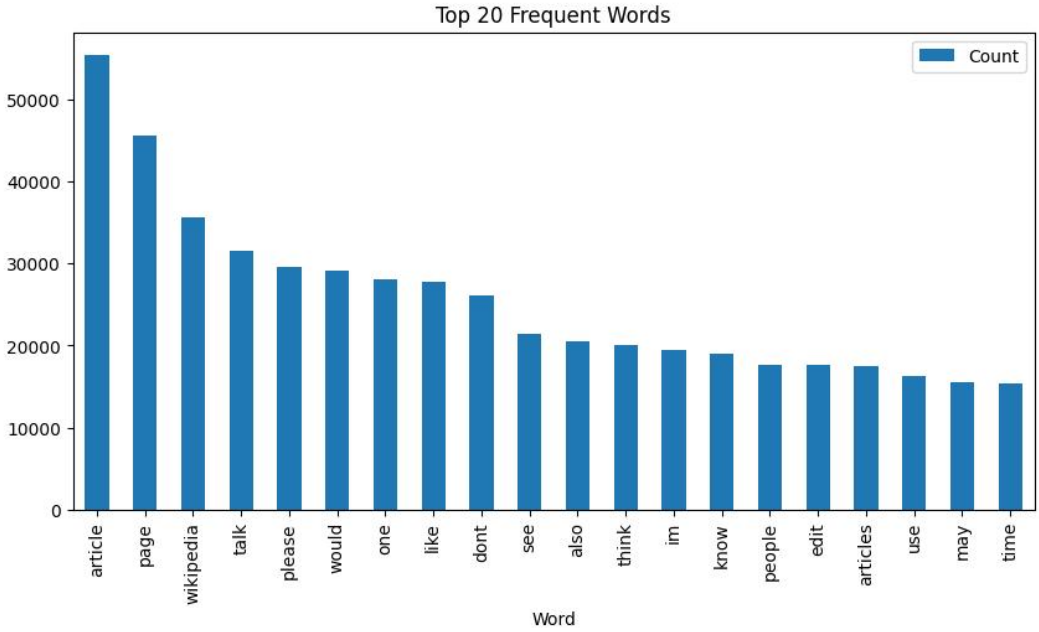
- Comment Length Distribution



This graph shows the distribution of comment lengths in the dataset.
Most comments are relatively short, while only a few comments contain very long text.
Understanding comment length helps in choosing suitable sequence length during text vectorization.

- Word cloud visualization





The non-toxic word cloud displays the most common words used in normal comments.

Compared to toxic comments, these words are generally more neutral and conversational.

This helps understand the difference in language patterns between toxic and non-toxic comments.

The word cloud visualizes the most frequently occurring words in toxic comments.

Words appearing larger indicate higher frequency.

This helps identify commonly used offensive or abusive words in toxic comments.

- Missing value analysis
- Duplicate value analysis

Observation from EDA

- The dataset was imbalanced.
- Approximately 89.8% comments were non-toxic.
- Around 10.2% comments were toxic.
- Categories such as toxic, obscene, and insult showed strong correlation.
- Categories like severe_threat and identity hate contained fewer samples.

Handling Imbalanced Dataset

The dataset was imbalanced, but balancing techniques such as SMOTE were not applied because:

- SMOTE is mainly suitable for numerical/tabular datasets.
- This project involved NLP text data and deep learning.
- The dataset size was sufficiently large.
- The model achieved good validation performance without balancing.

Class weighting could be considered in future improvements for better minority class handling.

Deep Learning Model

The model architecture used:

- Embedding Layer
- Bidirectional LSTM Layer
- Dense Layers
- Sigmoid Output Layer

Loss Function

Binary Crossentropy

Optimizer

Adam Optimizer

About LSTM and Bidirectional LSTM

Long Short-Term Memory (LSTM) is a type of Recurrent Neural Network (RNN) specially designed for processing sequential data such as text, speech, and time-series data. Traditional RNNs face difficulties in remembering long-term dependencies due to the vanishing gradient problem. LSTM overcomes this limitation by using memory cells and gating mechanisms that help retain important information for longer periods.

In Natural Language Processing (NLP), LSTM is widely used because it can understand the sequence and context of words in a sentence. This helps the model capture meaningful patterns in text data and improve prediction performance.

In this project, a Bidirectional LSTM (BiLSTM) model was used. Unlike a normal LSTM that processes text only in the forward direction, Bidirectional LSTM processes the text in both forward and backward directions. This allows the model to understand context from both previous and next words in a sentence.

For example, in toxic comment detection, the meaning of a word may depend on surrounding words. Bidirectional LSTM helps capture this contextual information more effectively, leading to better classification performance.

The deep learning model used in this project consists of:

- Embedding Layer
  Converts words into dense numerical vectors.
- Bidirectional LSTM Layer
  Learns contextual relationships from text sequences in both directions.
- Dense Layers
  Perform feature extraction and classification.

- Sigmoid Output Layer
  Predicts probabilities for multiple toxicity categories.

The Bidirectional LSTM model was chosen because it performs well in text classification tasks and can effectively learn complex language patterns from user comments.

```
1  model.summary()
```

```
Model: "sequential"
_________________________________________________________________
 Layer (type)                Output Shape              Param #
=================================================================
 embedding (Embedding)       (None, None, 32)          6400032

 bidirectional (Bidirection  (None, 64)                16640
 al)

 dense (Dense)               (None, 128)               8320

 dense_1 (Dense)             (None, 256)               33024

 dense_2 (Dense)             (None, 128)               32896

 dense_3 (Dense)             (None, 6)                 774

=================================================================
Total params: 6491686 (24.76 MB)
Trainable params: 6491686 (24.76 MB)
Non-trainable params: 0 (0.00 Byte)
_________________________________________________________________
```

Model Performance

The model achieved strong performance during training:

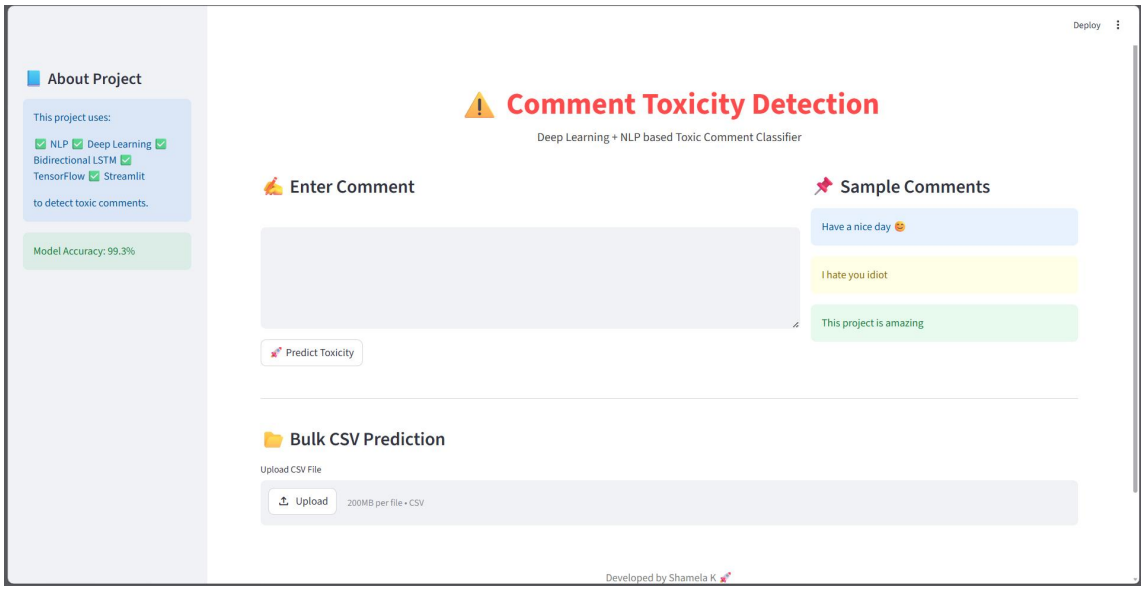
- **Training Accuracy: 99.33%**
- **Validation Accuracy: 99.30%**

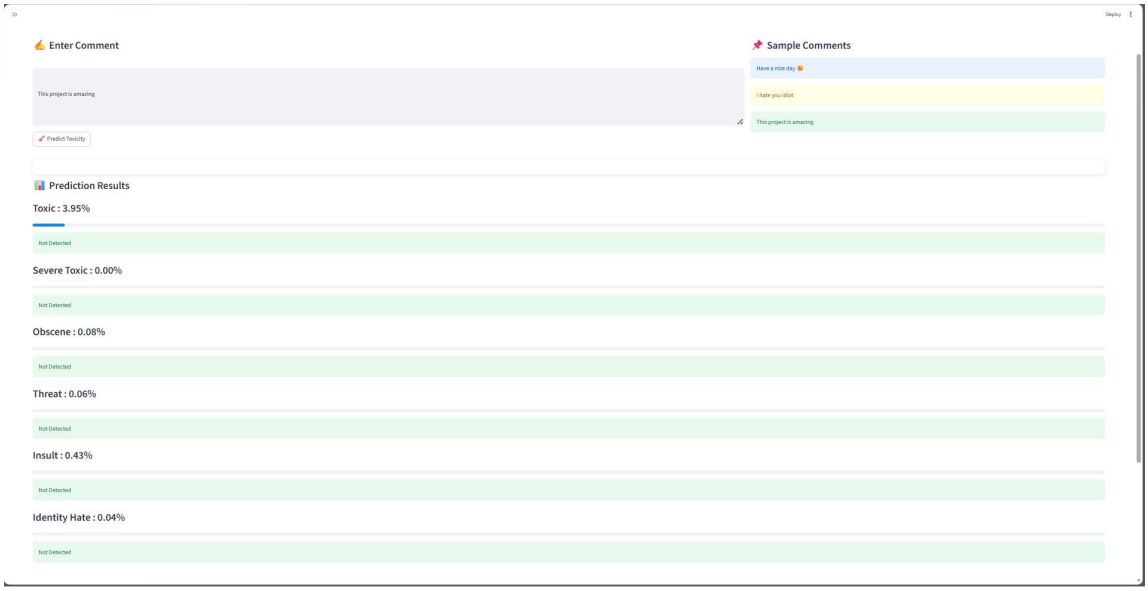
The loss values decreased during training, indicating improved prediction performance.

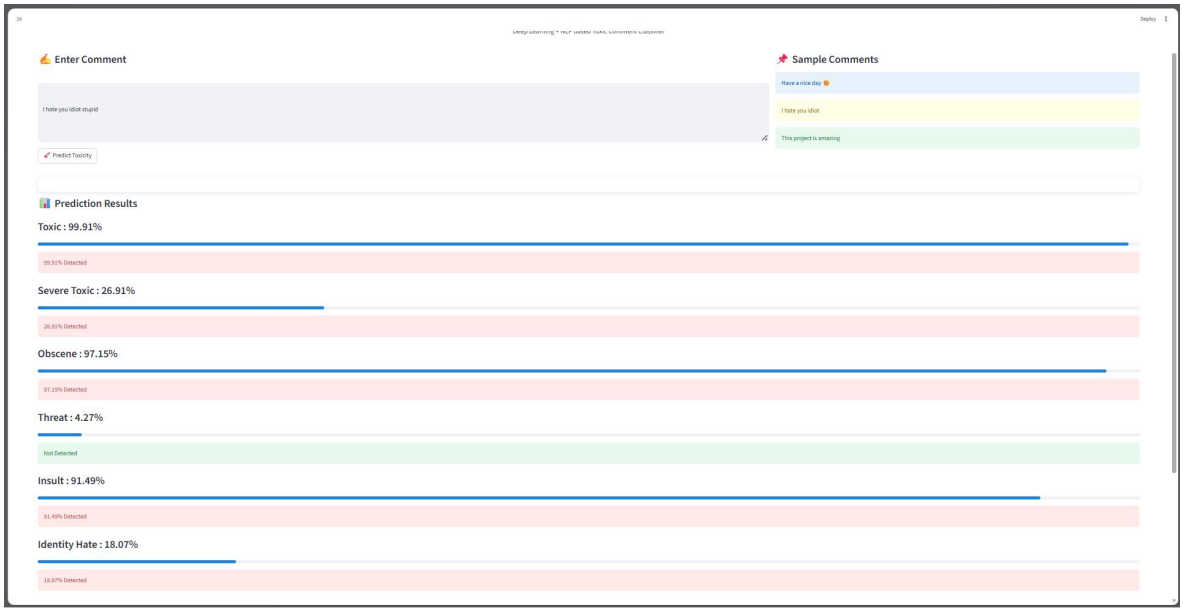
Streamlit Application

An interactive Streamlit application was developed with the following features:

- Real-time toxicity prediction
- CSV file upload for bulk prediction
- Display of prediction probabilities
- Interactive user interface
- Sample toxic comment testing







Technologies Used

- Python
- TensorFlow
- Keras
- NLP
- NLTK
- Pandas
- NumPy
- Matplotlib
- Seaborn
- Streamlit

Sample Prediction

Input Comment

I hate you idiot

Predicted Output

toxic : detected
insult : detected

Future Enhancements

Future improvements for this project may include:

- Using Transformer-based models such as BERT
- Applying class weighting techniques
- Improving prediction accuracy for minority classes
- Deploying the application on cloud platforms
- Supporting multilingual toxicity detection

Conclusion

This project successfully demonstrated how Natural Language Processing and Deep Learning can be used for toxic comment classification.

The Bidirectional LSTM model effectively learned text patterns and achieved high validation accuracy. The Streamlit application provides a user-friendly interface for real-time toxicity detection and bulk predictions.

This system can help online platforms automatically identify harmful comments and support safer digital communication environments.